CAFFÈ BELTRAMI

UNIVERSITÀ DEGLI STUDI DI PAVIA

Caffè Beltrami presenta…

Ivan Cucchi

Studente PhD in Computational Mathematics, Learning, and Data Science.
La mia ricerca si concentra sull'applicazione del Machine Learning e del Deep Learning nella Chimica Computazionale, con particolare attenzione al design e all'evoluzione di proteine.

CAFFÈ BELTRAMI

CAFFÈ BELTRAMI

UNIVERSITÀ DEGLI STUDI DI PAVIA

Black (box) Mirror: Tales from the Random Forest

Disclaimer

Disclaimer

Ogni riferimento ad algoritmi realmente esistenti
o aziende attualmente operative è puramente casuale…
con un'accuratezza del 92.3%.

The Netflix Prize

Il 2 ottobre 2006, Netflix lancia una sfida: la **Netflix Prize Competition**.

- **Obiettivo**: migliorare del 10% le prestazioni di Cinematch, il loro algoritmo di raccomandazione.
- **Premio: 1.000.000 $** al primo team che ci riesce + **50.000 $** al migliore ogni anno.

The Netflix Prize

Il 2 ottobre 2006, Netflix lancia una sfida: la **Netflix Prize Competition**.

- **Obiettivo**: migliorare del 10% le prestazioni di Cinematch, il loro algoritmo di raccomandazione.
- **Premio: 1.000.000 $** al primo team che ci riesce + **50.000 $** al migliore ogni anno.

Nei primi 6 giorni c'è già una squadra che migliora le prestazioni, al 15 ottobre le squadre sono 3, a fine mese più di 40.

The Netflix Prize

Il 2 ottobre 2006, Netflix lancia una sfida: la **Netflix Prize Competition**.

- **Obiettivo**: migliorare del 10% le prestazioni di Cinematch, il loro algoritmo di raccomandazione.
- **Premio: 1.000.000 $** al primo team che ci riesce + **50.000 $** al migliore ogni anno.

Nei primi 6 giorni c'è già una squadra che migliora le prestazioni, al 15 ottobre le squadre sono 3, a fine mese più di 40.

Netflix

The Netflix Prize

Il 2 ottobre 2006, Netflix lancia una sfida: la **Netflix Prize Competition**.

- **Obiettivo**: migliorare del 10% le prestazioni di Cinematch, il loro algoritmo di raccomandazione.
- **Premio: 1.000.000 $** al primo team che ci riesce + **50.000 $** al migliore ogni anno.

Nei primi 6 giorni c'è già una squadra che migliora le prestazioni, al 15 ottobre le squadre sono 3, a fine mese più di 40.

Netflix

Teams

Top20 Leaderboard

Rank	Team Name	Best Test Score	% Improvement	Best Submit Time
Grand Prize - RMSE = 0.8567 - Winning Team: BellKor's Pragmatic Chaos				
1	BellKor's Pragmatic Chaos	0.8567	10.06	2009-07-26 18:18:28
2	The Ensemble	0.8567	10.06	2009-07-26 18:38:22
3	Grand Prize Team	0.8582	9.90	2009-07-10 21:24:40
4	Opera Solutions and Vandelay United	0.8588	9.84	2009-07-10 01:12:31
5	Vandelay Industries !	0.8591	9.81	2009-07-10 00:32:20
6	PragmaticTheory	0.8594	9.77	2009-06-24 12:06:56
7	BellKor in BigChaos	0.8601	9.70	2009-05-13 08:14:09
8	Dace_	0.8612	9.59	2009-07-24 17:18:43
9	Feeds2	0.8622	9.48	2009-07-12 13:11:51
10	BigChaos	0.8623	9.47	2009-04-07 12:33:59
11	Opera Solutions	0.8623	9.47	2009-07-24 00:34:07
12	BellKor	0.8624	9.46	2009-07-26 17:19:11
Progress Prize 2008 - RMSE = 0.8627 - Winning Team: BellKor in BigChaos				
13	xiangliang	0.8642	9.27	2009-07-15 14:53:22
14	Gravity	0.8643	9.26	2009-04-22 18:31:32
15	Ces	0.8651	9.18	2009-06-21 19:24:53
16	Invisible Ideas	0.8653	9.15	2009-07-15 15:53:04
17	Just a guy in a garage	0.8662	9.06	2009-05-24 10:02:54
18	J Dennis Su	0.8666	9.02	2009-03-07 17:16:17
19	Craig Carmichael	0.8666	9.02	2009-07-25 16:00:54
20	acmehill	0.8668	9.00	2009-03-21 16:20:50
Progress Prize 2007 - RMSE = 0.8723 - Winning Team: KorBell				
Cinematch score - RMSE = 0.9525				

I team in testa alla competizione hanno spesso utilizzato a **metodi** di tipo **ensemble**, combinando algoritmi diversi come la decomposizione ai valori singolari (SVD), k-nearest neighbor (KNN), reti neurali (ANN) e altri modelli.

CAFFÈ BELTRAMI

Metodi Ensemble

Metodi ensemble

In Machine Learning, i **metodi ensemble** utilizzano più algoritmi di apprendimento per ottenere prestazioni predittive migliori di quelle che si potrebbero ottenere con uno qualsiasi degli algoritmi di apprendimento costituenti da solo.

Metodi Ensemble

Metodi ensemble

In Machine Learning, i **metodi ensemble** utilizzano più algoritmi di apprendimento per ottenere prestazioni predittive migliori di quelle che si potrebbero ottenere con uno qualsiasi degli algoritmi di apprendimento costituenti da solo.

Si divide in tre tecniche fondamentali:

- **Bagging**: combina modelli indipendenti e fa una "votazione democratica"; ogni modello ha lo stesso peso (es. Random Forest)
- **Boosting**: costruisce modelli in sequenza, dando più peso a quelli che correggono gli errori dei precedenti (es. Gradient Boosting, AdaBoost).
- **Stacking**: utilizza un *meta-modello* finale che impara a combinare le predizioni degli altri modelli.

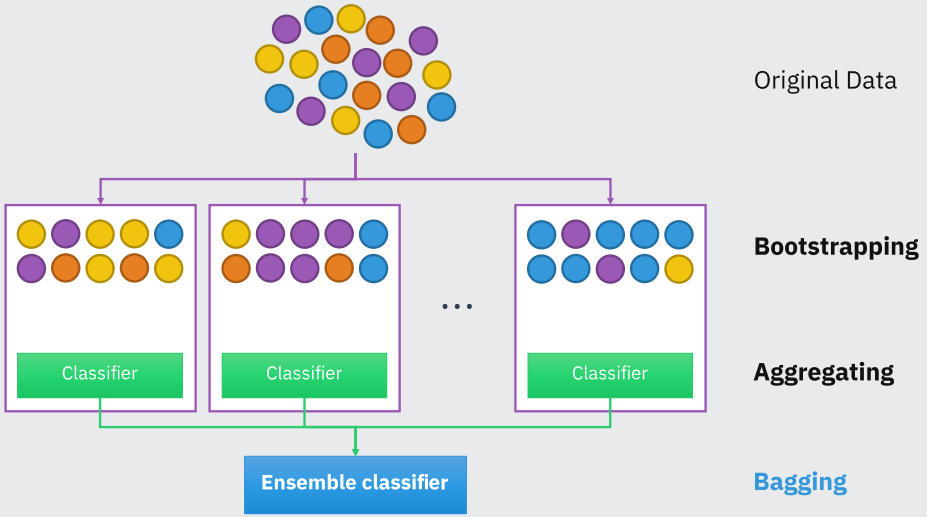
Metodi Ensemble

- **Bagging**: combina modelli indipendenti e fa una “votazione democratica”; ogni modello ha lo stesso peso (es. Random Forest)

Bagging

Bagging viene da **b**ootstrap **agg**regat**ing**, ovvero:

- **Bootstrap**: campionamento con reimmisione dal dataset originale.
- **Aggregating**: combinazione delle predizioni di tutti i modelli, in genere con una votazione di maggioranza.



CAFFÈ BELTRAMI

Bootstrap

Bootstrap

Sia $\mathcal{D}$ un dataset di n elementi. Il **bootstrap** genera B nuovi dataset $\mathcal{D}_i$ di n' elementi ciascuno, campionando con reimmissione da $\mathcal{D}$ con distribuzione uniforme.

N.B. Il campionamento con reimmisione permette elementi ripetuti in ogni $\mathcal{D}_i$; se $n = n'$, allora per n grandi si ha che ogni $\mathcal{D}_i$ ha in media circa il 63.2% di elementi unici ($\sim 1 - \frac{1}{e}$).

Questo garantisce che ogni set $\mathcal{D}_i$ sia indipendente dagli altri, poiché ogni campionamento è effettuato separatamente.

Dataset

Sia $\mathcal{D} = \left\{(\mathbf{x}_i, y_i) \mid (\mathbf{x}_i, y_i) \in \mathcal{X} \times \mathcal{Y}, i = 1, \ldots, n\right\}$ un dataset, dove

- $\mathbf{x}_i = (x_{ij})_{j=1,\ldots,m}$ è il vettore di **features** del campione i,
- y_i rappresenta la **classe** del campione i.
- $\mathcal{D} = \mathcal{A} \sqcup \mathcal{T} (\sqcup \mathcal{V})$, che sono i set di **addestramento**, **test** (e **validazione**).

Dataset

Sia $\mathcal{D} = \{(\mathbf{x}_i, y_i) \mid (\mathbf{x}_i, y_i) \in \mathcal{X} \times \mathcal{Y}, i = 1, \ldots, n\}$ un dataset, dove

- $\mathbf{x}_i = (x_{ij})_{j=1,\ldots,m}$ è il vettore di **features** del campione i,
- y_i rappresenta la **classe** del campione i.
- $\mathcal{D} = \mathcal{A} \sqcup \mathcal{T} (\sqcup \mathcal{V})$, che sono i set di **addestramento**, **test** (e **validazione**).

Generalmente, $\mathcal{X} = \mathbb{R}^m$ e $\mathcal{Y} = \{1, \ldots, l\}$, dove $l \in [2, +\infty)$ è il numero di classi.

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
0	0	0	1975	2295	7246	603	1711	2020	7	210	274	2577
1	1111	1	1091	4303	409	426	1706	2021	11	110	414	1762
2	2222	1	2651	2105	6296	736	1706	2021	11	0	242	7341
3	3333	1	3506	4303	7246	736	1706	2021	11	0	297	3617
4	4444	1	3861	4303	4815	251	1706	2021	11	110	393	4416
...	...	...	...	...	...	...	...	...	...	...	...	...
8802	8671	0	8770	979	4677	603	1419	2007	8	70	269	895
8803	8672	1	8773	4303	7246	736	788	2018	14	110	424	8483
8804	8673	0	8774	3631	3231	603	1366	2009	8	206	207	5228
8805	8674	0	8777	3247	7061	603	665	2006	6	206	125	3315
8806	8675	0	8781	2926	7298	251	1127	2015	9	16	328	1004

Classificatore

Classificatore

L'algoritmo di classificazione cerca di determinare la funzione, detta **classificatore**, che meglio approssima la relazione che esiste tra i dati, scegliendo nello spazio delle ipotesi

$$\mathsf{H} = \left\{ \mathsf{h}_\theta : \mathcal{X} \to \mathcal{Y} \mid \theta \in \Theta \right\},$$

dove Θ è un insieme di **parametri** finito.

Classificatore

Classificatore

L'algoritmo di classificazione cerca di determinare la funzione, detta **classificatore**, che meglio approssima la relazione che esiste tra i dati, scegliendo nello spazio delle ipotesi

$$H = \{h_\theta : \mathcal{X} \to \mathcal{Y} \mid \theta \in \Theta\},$$

dove Θ è un insieme di **parametri** finito.

Per decidere l'h_θ migliore, viene utilizzata una **loss function** $\mathcal{L}$. Posto $\hat{y}_i = h_\theta(\mathbf{x}_i)$ la classe predetta da h_θ per l'i-esimo dato, la loss più semplice è:

$$\mathcal{L} : \mathcal{Y} \times \mathcal{Y} \to \{0, 1\} \quad \text{definita da} \quad \mathcal{L}(\hat{y}_i, y_i) = \begin{cases} 1 & \hat{y}_i \neq y_i \\ 0 & \hat{y}_i = y_i \end{cases}.$$

Albero decisionale: costruzione

Un albero decisionale su $\mathcal{X}$ è un classificatore non parametrico con una struttura di albero radicato, binario, finito T in cui

Albero decisionale: costruzione

Un albero decisionale su $\mathcal{X}$ è un classificatore non parametrico con una struttura di albero radicato, binario, finito T in cui

- ogni nodo v $\in$ T ha un solo nodo genitore, ad eccezione del nodo radice r,

Albero decisionale: costruzione

Un albero decisionale su $\mathcal{X}$ è un classificatore non parametrico con una struttura di albero radicato, binario, finito T in cui

- ogni nodo $v \in T$ ha un solo nodo genitore, ad eccezione del nodo radice r,
- ogni nodo $v \in T$ ha esattamente due nodi figli v0 e v1 (**interno**) oppure nessuno (**foglia**) e appartiene a $T_f \subset T$,

Albero decisionale: costruzione

Un albero decisionale su $\mathcal{X}$ è un classificatore non parametrico con una struttura di albero radicato, binario, finito T in cui

- ogni nodo $v \in T$ ha un solo nodo genitore, ad eccezione del nodo radice r,
- ogni nodo $v \in T$ ha esattamente due nodi figli v0 e v1 (**interno**) oppure nessuno (**foglia**) e appartiene a $T_f \subset T$,
- ogni nodo $v \in T$ è associato ad una regione $R_v \subseteq \mathcal{X}$; per $v = r$ si ha $R_r = \mathcal{X}$, mentre $\forall v$ interno si ha che R_v è un elemento della partizione della regione associata al suo nodo genitore.

Albero decisionale: costruzione

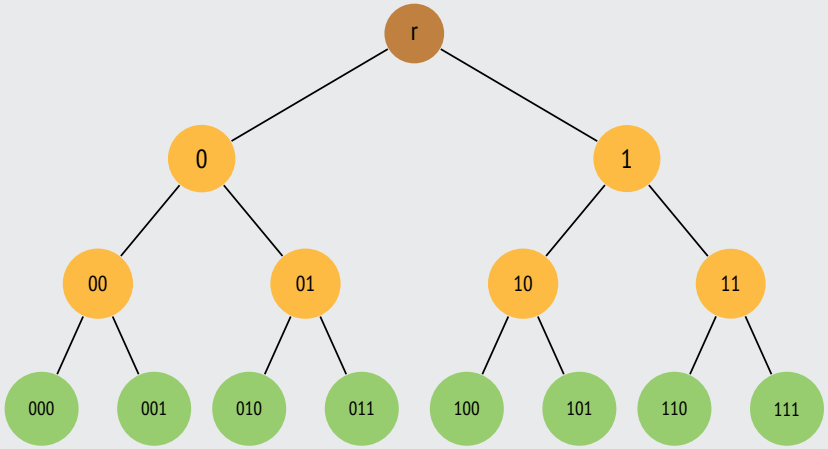
Un albero decisionale su $\mathcal{X}$ è un classificatore non parametrico con una struttura di albero radicato, binario, finito T in cui

- ogni nodo $v \in T$ ha un solo nodo genitore, ad eccezione del nodo radice r,
- ogni nodo $v \in T$ ha esattamente due nodi figli v0 e v1 (**interno**) oppure nessuno (**foglia**) e appartiene a $T_f \subset T$,
- ogni nodo $v \in T$ è associato ad una regione $R_v \subseteq \mathcal{X}$; per $v = r$ si ha $R_r = \mathcal{X}$, mentre $\forall v$ interno si ha che R_v è un elemento della partizione della regione associata al suo nodo genitore.

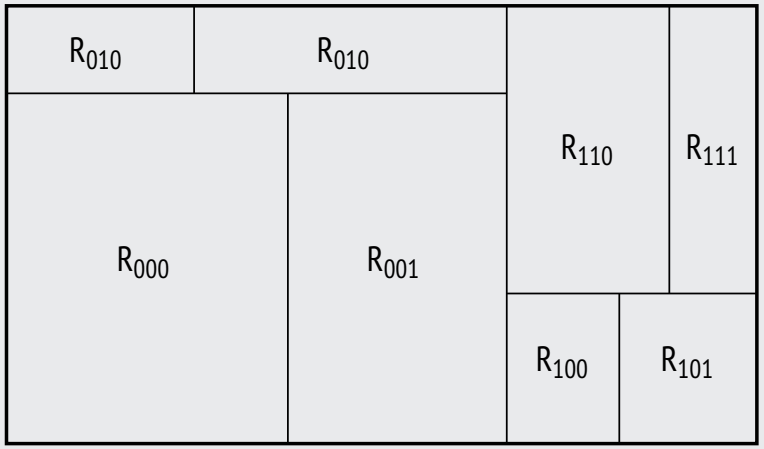
Dato $v \in T \setminus T_f$ e denotati con $a_v \in \{1, \ldots, m\}$ e b_v rispettivamente la dimensione e la posizione del taglio di R_v, si determina una partizione $\{R_{v0}, R_{v1}\}$ della regione, dove

$$R_{v0} = \{\mathbf{x} \in R_v \mid x_{a_v} \leq b_v\} \quad \text{e} \quad R_{v1} = \{\mathbf{x} \in R_v \mid x_{a_v} > b_v\}.$$

Albero decisionale: rappresentazione



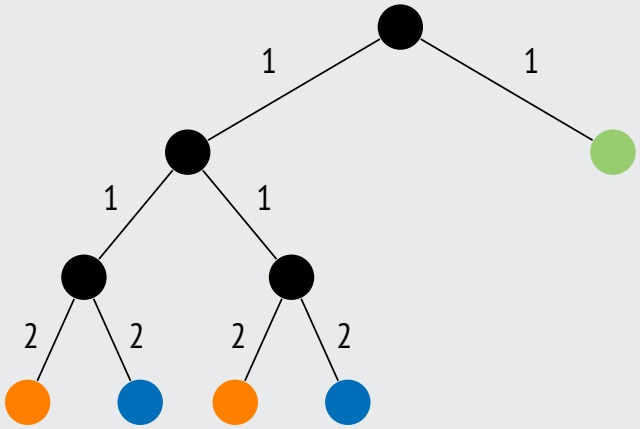
Albero decisionale



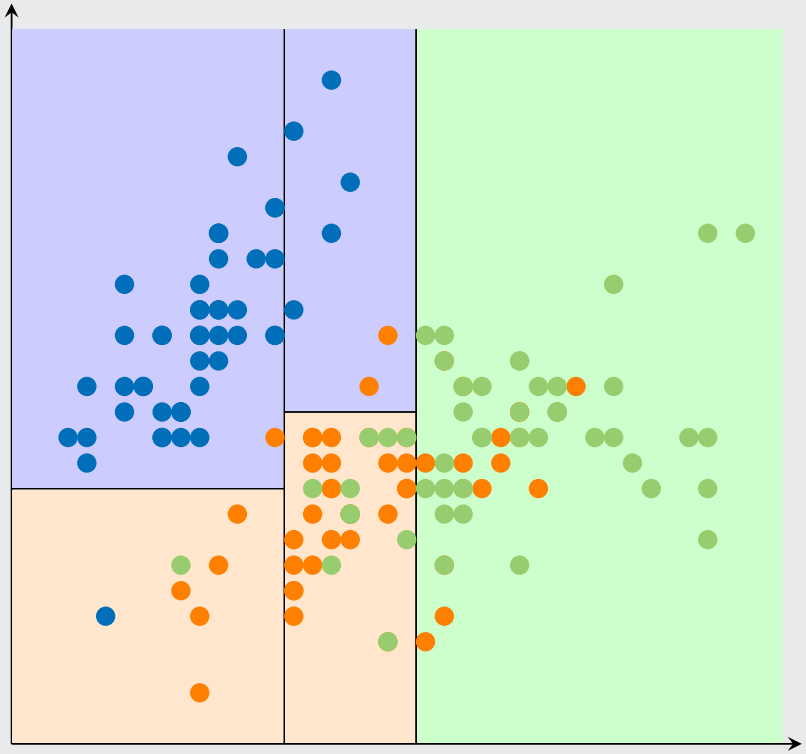
Partizione di $\mathcal{X}$

CAFFÈ BELTRAMI

Albero decisionale: rappresentazione



Albero decisionale



Partizione di $\mathcal{X}$

CAFFÈ BELTRAMI

Albero decisionale: costruzione

L'algoritmo determina la migliore partizione attraverso l'ottimizzazione di criteri di divisione, dipendenti da $\hat{p}_{vc} = \frac{1}{n_v} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{A} \cap R_v} \delta(y_i, c)$. I più utilizzati sono:

- **Tasso d'errore**. $E = \frac{1}{n_v} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{A} \cap R_v} \Big(1 - \delta(y_i, c)\Big) = 1 - \hat{p}_{vc_v}$;
- **Indice di Gini**. $G = \sum_{c \neq \tilde{c}} \hat{p}_{vc} \hat{p}_{v\tilde{c}} = \sum_{c=1}^{l} \hat{p}_{vc}(1 - \hat{p}_{vc})$;
- **Variazione di entropia**. $D = -\sum_{c=1}^{l} \hat{p}_{vc} \log \hat{p}_{vc}$.

CAFFÈ BELTRAMI

Albero decisionale: costruzione

L'algoritmo determina la migliore partizione attraverso l'ottimizzazione di criteri di divisione, dipendenti da $\hat{p}_{vc} = \frac{1}{n_v} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{A} \cap R_v} \delta(y_i, c)$. I più utilizzati sono:

- **Tasso d'errore**. $E = \frac{1}{n_v} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{A} \cap R_v} \Big(1 - \delta(y_i, c)\Big) = 1 - \hat{p}_{vc_v}$;
- **Indice di Gini**. $G = \sum_{c \neq \tilde{c}} \hat{p}_{vc} \hat{p}_{v\tilde{c}} = \sum_{c=1}^{l} \hat{p}_{vc}(1 - \hat{p}_{vc})$;
- **Variazione di entropia**. $D = -\sum_{c=1}^{l} \hat{p}_{vc} \log \hat{p}_{vc}$.

Successivamente, per trovare la profondità ottimale dell'albero si usa una strategia di **potatura**, che consiste nel trovare $\alpha \geq 0$ che minimizzi la quantità $C_\alpha(T) = \sum_{v=1}^{|T|} n_v Q_v(T) + \alpha |T|$.

CAFFÈ BELTRAMI

Albero decisionale

Albero decisionale

Un **albero decisionale** è definito come il dato $(T, \mathbf{a}, \mathbf{b})$, dove $\mathbf{a} = (a_v)_{v \in T \setminus T_f}$ e $\mathbf{b} = (b_v)_{v \in T \setminus T_f}$; le regioni associate ai nodi foglia di T_f rappresentano una partizione dell'intero spazio delle caratteristiche $\mathcal{X}$.

Preso $\mathbf{z} \in \mathcal{T}$, si ha che $\mathbf{z} \in R_v$ per un qualche nodo foglia $v \in T_f$; l'algoritmo assegna a $\mathbf{z}$ la classe di maggioranza tra quelle dei campioni di $\mathcal{A}$ contenuti nella stessa regione R_v, definita da

$$\hat{y}_{\mathbf{z}} = \arg\max_{c \in \mathcal{Y}} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{A} \cap R_v} \delta(y_i, c).$$

CAFFÈ BELTRAMI

ATTENZIONE

Gli alberi decisionali molto profondi tendono ad avere una varianza elevata, adattandosi troppo ai dati di addestramento (**OVERFITTING!**)

Random Forest

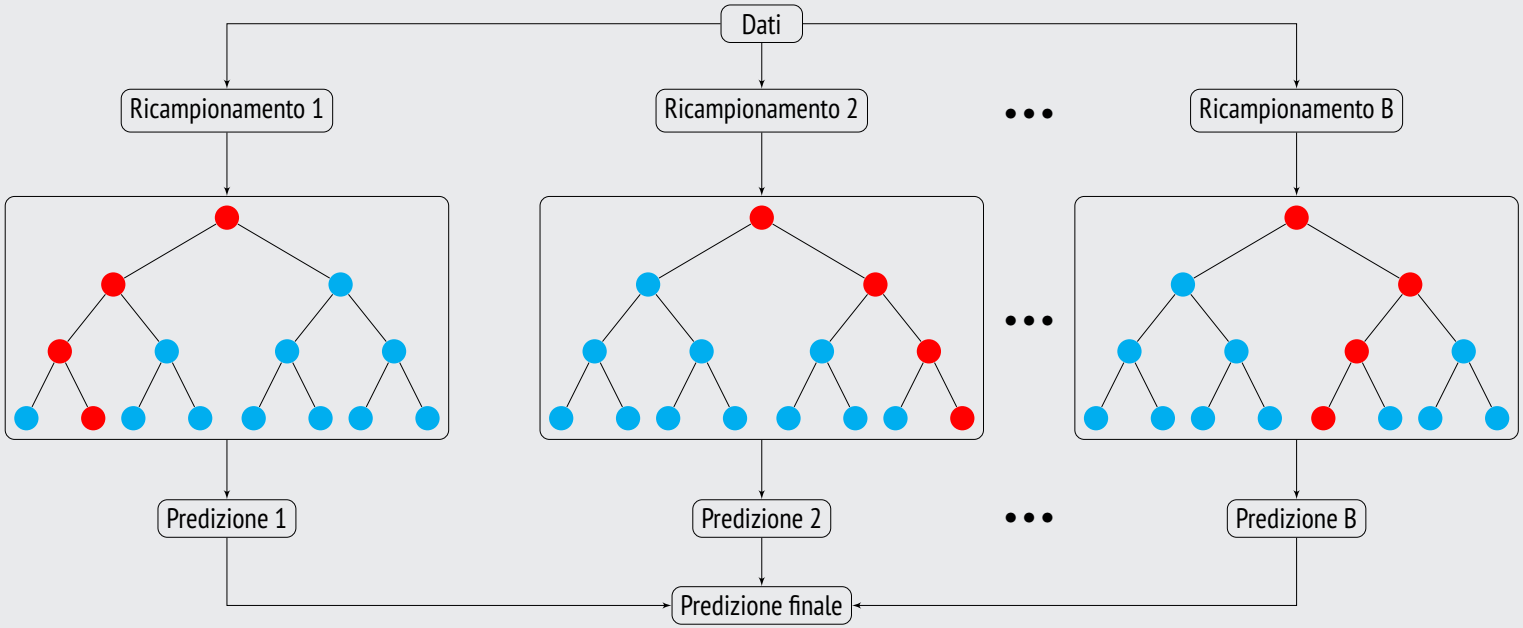
Random Forest

Il **Random Forest** è un classificatore ottenuto dall'aggregazione tramite **bagging** di **alberi decisionali**.

Preso $\mathbf{z} \in \mathcal{T}$, l'algoritmo gli assegna la classe di maggioranza tra le $\hat{y}^r_{\mathbf{z}}$ determinate da ogni albero decisionale T_r, per $r = 1, \ldots, B$, definita da

$$\hat{y}_{\mathbf{z}} = \arg\max_{r=1,\ldots,B} \hat{y}^r_{\mathbf{z}}.$$

Random Forest: rappresentazione



CAFFÈ BELTRAMI

Random Forest: costruzione

Date $t_1, \ldots, t_B$ variabili identicamente distribuite con indice di correlazione $\rho > 0$ e varianza σ^2, si ha che la varianza della media $\bar{t}$ è $\sigma^2_{\bar{t}} = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$.

Random Forest: costruzione

Date $t_1, \ldots, t_B$ variabili identicamente distribuite con indice di correlazione $\rho > 0$ e varianza σ^2, si ha che la varianza della media $\bar{t}$ è $\sigma^2_{\bar{t}} = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$.

L'obiettivo del Random Forest è minimizzare $\sigma^2_{\bar{t}}$ decorrelando gli alberi: prima di ciascuna partizione, vengono selezionate casualmente $p \leq m$ (solitamente $p = \sqrt{m}$) features su cui basare il criterio di split.

Esempio: Netflix dataset

Campo	Descrizione
`show_id`	Identificatore univoco del film o della serie
`type`	Tipo di contenuto: “Movie” o “TV Show”
`title`	Titolo del film o della serie
`director`	Regista (può essere mancante)
`cast`	Attori del cast (può essere mancante)
`country`	Paese di produzione o associato
`date_added`	Data di aggiunta alla piattaforma
`release_year`	Anno di uscita originale
`rating`	Classificazione per età (es. “PG-13”, “TV-MA”)
`duration`	Durata: in minuti per film, in stagioni per serie
`listed_in`	Genere o categoria (es. “Documentari”)
`description`	Breve descrizione o trama

Esempio: Netflix dataset

Classificatori:

- **Albero decisionale**: `criterion` = gini, `ccp_alpha` = 0, `random_state` = 42
- **Random Forest**: `n_estimators` = 10/100, `criterion` = gini, `max_features` = sqrt, `bootstrap` = True, `ccp_alpha` = 0, `random_state`: 42

Dataset: $\mathcal{D} = \mathcal{A} \sqcup \mathcal{T}$, con $n_{\mathcal{A}} = 0.8 \cdot n$ e $n_{\mathcal{T}} = 0.2 \cdot n$

Esempio: Netflix dataset

Classificatori:

- **Albero decisionale**: `criterion` = gini, `ccp_alpha` = 0, `random_state` = 42
- **Random Forest**: `n_estimators` = 10/100, `criterion` = gini, `max_features` = sqrt, `bootstrap` = True, `ccp_alpha` = 0, `random_state`: 42

Dataset: $\mathcal{D} = \mathcal{A} \sqcup \mathcal{T}$, con $n_{\mathcal{A}} = 0.8 \cdot n$ e $n_{\mathcal{T}} = 0.2 \cdot n$

Risultati:

- Albero decisionale: ACC = 0.9886

CAFFÈ BELTRAMI

Esempio: Netflix dataset

Classificatori:

- **Albero decisionale**: `criterion` = gini, `ccp_alpha` = 0, `random_state` = 42
- **Random Forest**: `n_estimators` = 10/100, `criterion` = gini, `max_features` = sqrt, `bootstrap` = True, `ccp_alpha` = 0, `random_state`: 42

Dataset: $\mathcal{D} = \mathcal{A} \sqcup \mathcal{T}$, con $n_{\mathcal{A}} = 0.8 \cdot n$ e $n_{\mathcal{T}} = 0.2 \cdot n$

Risultati:

- Albero decisionale: ACC = 0.9886
- Random Forest (10): ACC = 0.9926

Esempio: Netflix dataset

Classificatori:

- **Albero decisionale**: `criterion` = gini, `ccp_alpha` = 0, `random_state` = 42
- **Random Forest**: `n_estimators` = 10/100, `criterion` = gini, `max_features` = sqrt, `bootstrap` = True, `ccp_alpha` = 0, `random_state`: 42

Dataset: $\mathcal{D} = \mathcal{A} \sqcup \mathcal{T}$, con $n_{\mathcal{A}} = 0.8 \cdot n$ e $n_{\mathcal{T}} = 0.2 \cdot n$

Risultati:

- Albero decisionale: ACC = 0.9886
- Random Forest (10): ACC = 0.9926
- Random Forest (100): ACC = **0.9943**

Take Home Message

- **Bagging** non è una canzone.

CAFFÈ BELTRAMI

Take Home Message

- **Bagging** non è una canzone. È un metodo ensemble che combina modelli indipendenti tramite bootstrap.

Take Home Message

- **Bagging** non è una canzone. È un metodo ensemble che combina modelli indipendenti tramite bootstrap.
- L'**albero decisionale** è un classificatore non parametrico con struttura ad albero

Take Home Message

- **Bagging** non è una canzone. È un metodo ensemble che combina modelli indipendenti tramite bootstrap.
- L'**albero decisionale** è un classificatore non parametrico con struttura ad albero: potiamolo!

Take Home Message

- **Bagging** non è una canzone. È un metodo ensemble che combina modelli indipendenti tramite bootstrap.
- L'**albero decisionale** è un classificatore non parametrico con struttura ad albero: potiamolo!
- "More classifiers is megl' che uan".

Take Home Message

- **Bagging** non è una canzone. È un metodo ensemble che combina modelli indipendenti tramite bootstrap.
- L'**albero decisionale** è un classificatore non parametrico con struttura ad albero: potiamolo!
- "More classifiers is megl' che uan". **Random Forest** ha prestazioni migliori rispetto ad un singolo albero decisionale.

Bibliografia

- Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). Classification and Regression Trees. Monterey, CA: Wadsworth International Group.
- Breiman, L. (2001). Random Forests. Machine Learning, **45**(1), 5–32.
- Shivam Bansal. (2021). Netflix Movies and TV Shows. Kaggle. Disponibile su: `https://www.kaggle.com/datasets/shivamb/netflix-shows`

CAFFÈ BELTRAMI

Grazie peR l'attenzione!

CafFè Beltrami

CAFFÈ BELTRAMI